풀스택 개발자 양성과정

인터페이스 구현

1조 정승원, 김선길, 박형규, 박한결

목차

INDEX

01 | COMMON — JDBC 공통 기능 모음 (DB 연결, SHA-256, JSON 처리, UTF-8 인코딩 설정)

02 | MONITOR — 인터페이스 처리 결과 조회 및 실패 데이터 재시도 관리

03 | RECV — Outbox 데이터를 받아 계정 생성 / Inbox 기록 / 배치 처리 하는 수신 영역

04 | SEND — 사원 등록 후 Employee + Outbox 저장하는 송신 영역

01 COMMON - DBManager(송신)

```java
/** 송신측 HrmDB Connection */
public static Connection getHrmConnection() throws SQLException {
    // TODO :
    String driver   = "com.mysql.cj.jdbc.Driver";
    String url      = "jdbc:mysql://localhost:3306/HrmDB?useSSL=false&allowPublicKeyRetrieval=true&se
    String user     = "ifuserx";
    String password = "ifx1234!";
    // db.properties에 입력한 driver + hrm.url + hrm.user+ hrm.password 값을 각 매개변수에 담기

    try {
        Class.forName(driver);
    } catch (ClassNotFoundException e) {
        throw new RuntimeException("DBManager.getHrmConnection Class.forName..." + e.getCause());
    }
    return DriverManager.getConnection(url, user, password);
    // getHrmConnection 클래스 실행되면 JDBC 드라이버에 (url,user,password) 접속 + 런타임에러 예외처리 + 에러원인
```

1. db.properties에 입력한 driver + hrm.url + hrm.user+ hrm.password 값을 각 매개변수에 담기
2. getHrmConnection 클래스 실행되면 JDBC 드라이버에 (url,user,password) 접속 + 런타임 에러 예외처리 + 에러원인

01 COMMON - DBManager(수신)

```java
/** 수신측 GroupwareDB Connection */
public static Connection getGroupwareConnection() throws SQLException {
    // TODO
    String driver   = "com.mysql.cj.jdbc.Driver";
    String url      = "jdbc:mysql://localhost:3306/GroupwareDB?useSSL=false&allowPublicKeyRetrieval=true&s
    String user     = "ifuserx";
    String password = "ifx1234!";
    // db.properties에 입력한 driver + gw.url + gw.user+ gw.password 값을 각 매개변수에 담기

    try {
        Class.forName(driver);
    } catch (ClassNotFoundException e) {
        throw new RuntimeException("DBManager.getGroupwareConnection Class.forName..." + e.getCause());
    }
    return DriverManager.getConnection(url, user, password);
    // getGroupwareConnection 클래스 실행되면 JDBC 드라이버에 (url,user,password) 접속 + 런타임에러 예외처리 + 에러원인
}
```

1. db.properties에 입력한 driver + gw.url + gw.user+ gw.password 값을 각 매개변수에 담기
2. getGroupwareConnection 클래스 실행되면 JDBC 드라이버에 (url,user,password) 접속 + 런타임 에러 예외처리 + 에러원인 확인

01 COMMON - DBManager(기본값)

```
public static int    getFetchSize()          { return 100; }
/* TODO : 한 번에 처리할 수 있는 행의 수 설정 */
public static int    getMaxRetry()           { return 3;   }
/* TODO : 재시도 횟수 설정 */

/** 계정 초기 비밀번호 suffix — empId + 이 값을 SHA-256 해시한 게 pwd_hash */
public static String getInitPasswordSuffix() { return "1234"; }
/* TODO : 초기 비밀번호 값 설정 */

/** AutoCloseable 자원 여러 개 안전 close */
public static void close(AutoCloseable... closeables) {
    /* TODO : 연결 종료 후 JDBC driver 자동으로 종료 */
    for (AutoCloseable c : closeables) {
        if (c != null) {
            try { c.close(); }
            catch (Exception e) { e.printStackTrace(); }
        }
    }
    // close() 처리가 되지 않을시 예외처리
```

1. getFetchSize() : 한 번에 처리할 수 있는 행의 수 설정
2. getMaxRetry() : 재시도 횟 수 값 설정
3. getInitPasswordSuffix() : 초기 비밀번호 값 설정
4. close : 연결 종료 후 JDBC driver 자동 종료

01 COMMON - HashUtil(보안)

```java
/** SHA-256 hex string */
public static String sha256(String plain) {
    // TODO: java.security.MessageDigest + UTF-8 + hex 변환
    if (plain == null) return null;
    // plain 값이 null 이라면 null을 줘라.
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        // md라는 SHA-256 객체 생성
        byte[] hash = md.digest(plain.getBytes(StandardCharsets.UTF_8));
        // plain을 받아 바이트로 변환시켜 hash에 저장 - hash 값 생성
        StringBuilder sb = new StringBuilder(64);
        // 64자 길이를 담을수 있는 sb객체 생성
        for (byte b : hash) sb.append(String.format("%02x", b));
        // hash 값을 꺼내 16진수로 치환 후 sb에 추가
        return sb.toString();
        // sb를 string 타입으로 리턴한다.
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException(e);
    } // 마지막 예외처리
```

1. plain 값이 null 이라면 null을 줘라.
2. SHA-256 객체 생성
3. hash값 생성
4. sb객체 생성
5. hash값 16진수 치환 후 sb에 추가
6. sb를 string 타입으로 리턴

02 MONITOR - IfMonitorServlet(객체 생성)

```java
//-----------------------------------------------------------------
//발신함(Outbox), 수신함(Inbox), 계정(Account) 데이터를 한 번에 조회하여
//"/monitor/list.jsp" 뷰로 전달하는 MVC 패턴의 컨트롤러
//-----------------------------------------------------------------

@WebServlet("/monitor/list_")   // "/monitor/list_" URL로 요청 보내면 이 서블릿이 실행되도록 매핑
public class IfMonitorServlet extends HttpServlet {

    // 데이터베이스 연동 처리할 DAO 객체 선언
    private OutboxDAO  outboxDAO;   // 발신함(Outbox) 데이터 조회 DAO
    private InboxDAO   inboxDAO;    // 수신함(Inbox) 데이터 조회 DAO
    private AccountDAO accountDAO;  // 계정(Account) 목록 조회 DAO

    // 서블릿 초기화. 사용할 DAO 객체들을 미리 생성
    @Override
    public void init() { // init(): 서블릿이 메모리에 로드될 때 최초 1회 실행되는 초기화 메서드
        outboxDAO  = new OutboxDAO();   // 발신함(Outbox) DAO 인스턴스 생성
        inboxDAO   = new InboxDAO();    // 수신함(Inbox) DAO 인스턴스 생성
        accountDAO = new AccountDAO();  // 계정(Account) DAO 인스턴스 생성
    }
```

1. URL요청 보내면 실행되게 서블릿 매핑
2. 데이터 연동할 DAO 객체 outboxDAO, inboxDAO, accountDAO 선언
3. init()으로 서블릿 초기화. 세 DAO 인스턴스 미리 생성

02 | MONITOR - IfMonitorServlet(요청 처리)

```
// HTTP POST 요청 처리 메서드
// 브라우저가 GET /monitor/list_ 요청 시 호출
// 3가지 목록 데이터를 DB에서 읽어 request에 담아 JSP 포워딩
@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
    // TODO
    System.out.println("GET /monitor/list");    // 서버 콘솔에 요청 수신 로그 출력

    try {
        // 발신함 전체 목록 조회. listAll()이 반환한 List를 "outboxList" 키로 request에 저장
        req.setAttribute("outboxList",  outboxDAO.listAll());
        // 수신함 전체 목록 조회. listAll()이 반환한 List를 "inboxList" 키로 request에 저장.
        req.setAttribute("inboxList",   inboxDAO.listAll());
        // 계정 전체 목록 조회. listAccounts()가 반환한 List를 "accountList" 키로 저장.
        req.setAttribute("accountList", accountDAO.listAccounts());
    } catch (SQLException e) {
        // DB 오류 처리 서버 콘솔 출력.
        e.printStackTrace();
        // 오류 메시지를 클라이언트에게 전송.
        req.setAttribute("error", "조회 오류: " + e.getMessage());
    }
    // 화면전환(Forward). 데이터를 담은 request(req)와 response(resp) 객체를
    // 가지고 지정된 JSP 페이지로 이동.
    // 브라우저 URL 바뀌지 않고 request 속성 그대로 유지
    req.getRequestDispatcher("/monitor/list.jsp").forward(req, resp);
}
```

1. println으로 요청 수신 로그 콘솔 출력
2. listAll()로 outboxDAO, inboxDAO, accountDAO의 모든 List 목록 조회
3. try catch로 오류 발생 시 콘솔 출력 & 클라이언트에게 전송
4. 화면전환(Forward). 데이터 담아 지정된 JSP 페이지로 이동. URL 바뀌지 않음

02 MONITOR - IfRetryServlet(객체 생성)

```java
//-----------------------------------------------------------------
//실패한 발신 메시지(Outbox)의 상태를 '재시도(F→N)' 가능하도록 변경하고
//모니터링 목록 페이지로 리다이렉트(Redirect)하는 MVC 패턴의 컨트롤러
//-----------------------------------------------------------------

@WebServlet("/monitor/retry_")  // "/monitor/retry_" URL로 요청 보내면 이 서블릿이 실행되도록 매핑
public class IfRetryServlet extends HttpServlet {

    // 데이터베이스 연동 처리할 OutboxDAO 객체 선언
    private OutboxDAO outboxDAO;

    // 서블릿 초기화. 사용할 OutboxDAO 객체 미리 생성
    @Override
    public void init() { // init(): 서블릿이 메모리에 로드될 때 최초 1회 실행되는 초기화 메서드
        outboxDAO = new OutboxDAO();
    }
```

1. URL요청 보내면 실행되게 서블릿 매핑 데이터 연동할 DAO 객체 outboxDAO 선언
2. init()으로 서블릿 초기화. DAO 인스턴스 미리 생성

02 MONITOR - IfRetryServlet(파라미터검증)

```java
// HTTP GET 요청 처리 메서드
// JSP/HTML 폼에서 outboxId를 POST로 전송 시 호출
// 유효성 검사 → DB 상태 변경 → 목록 페이지로 리다이렉트 진행
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
    // TODO
    System.out.println("POST /monitor/retry");  // 서버 콘솔에 요청 수신 로그 출력


    String idStr = req.getParameter("outboxId");   // 요청 파라미터 "outboxId" 추출. 없으면 null 반환

    // 파라미터 존재 여부 검증
    // outboxId가 없으면 Error 400 응답하고 종료. return으로 이후 로직 실행 막음
    if (idStr == null) { resp.sendError(400, "outboxId 필요"); return; }
```

1. println으로 요청 수신 로그 콘솔 출력
2. getParameter()로 요청 파라미터 "outboxId" 추출
3. if문과 sendError()로 파라미터 존재 여부 검증, null이면 Error400 응답하고 종료

02 MONITOR - IfRetryServlet(요청 처리)

```
        try {
            // Long.parseLong(idStr): 문자열 -> Long 변환. DB의 PK가 Long형이므로 문자열 파싱 필요
            // OutboxDAO.reopen(): outboxID 레코드 상태를 'F'(실패)에서 'N'(대기)으로 업데이트.
            outboxDAO.reopen(Long.parseLong(idStr));

        } catch (Exception e) {
            e.printStackTrace();  // DB 오류 처리 서버 콘솔 출력.
            resp.sendError(500, "재시도 오류: " + e.getMessage()); // 오류 메시지를 클라이언트에게 전송.

            return;
        }
        // 화면전환(Redirect). contextPath 경로를 얻어 지정된 URL(/monitor/list)로 페이지 이동.
        // 브라우저 URL이 바뀌어 새롭게 GET 요청을 보냄
        // req.getContextPath() : 경로가 없으면 빈 문자열 반환.
        resp.sendRedirect(req.getContextPath() + "/monitor/list");
    }
```

1. parseLong()으로 문자열→ Long 변환
2. OutboxDAO.reopen()으로 outboxID 레코드 상태를 'F'(실패)에서 'N'(대기)으로 업데이트
3. try catch로 오류 발생 시 콘솔 출력 & 클라이언트에게 전송
4. 화면전환(Redirect). contextPath 경로를 얻어 지정된 URL 페이지로 이동. URL이 바뀌어 새 GET 요청 보냄

03 RECV - AccountBatchServlet(Pending)

```
    //TODO : 배치 시작 -> pending 조회 -> processOne() 반복 처리 -> 통계 누적 -> JSP forward
    System.out.println("/recv/batch 배치 시작");

    int totalRead = 0, success = 0, fail = 0, skip = 0;
    StringBuilder log = new StringBuilder();

    try {
        List<Outbox> pendings = outboxDAO.fetchPending(DBManager.getMaxRetry(), DBManager.getFetchSize());
        totalRead = pendings.size();

        for (Outbox o : pendings) {
            String result = processOne(o);
            log.append(String.format("[outboxId=%d, tx=%s] %s%n", o.outboxId, o.txNo, result));
            if      (result.startsWith("OK"))   success++;
            else if (result.startsWith("SKIP")) skip++;
            else                                fail++;
        }
    } catch (Exception e) {
        e.printStackTrace();
        log.append("FATAL: ").append(e.getMessage()).append('\n');
    }

    req.setAttribute("totalRead", totalRead);
    req.setAttribute("success",   success);
    req.setAttribute("fail",      fail);
    req.setAttribute("skip",      skip);
    req.setAttribute("log",       log.toString());
    req.getRequestDispatcher("/recv/batchResult.jsp").forward(req, resp);
}
```

1. 처리 대기(Pending) 상태의 Outbox 데이터를 조회한 뒤, processOne()으로 각 데이터를 순차 처리

2. 처리 결과(OK / SKIP / FAIL)를 판별하여 성공·실패·스킵 횟수와 실행 로그 누적

3. 최종 집계 결과를 request에 저장하고 batchResult.jsp로 전달하여 화면에 출력

03 RECV - AccountBatchServlet-1(정상 처리)

```
// TODO :
// processOne 구현()
// 1) payload JSON 파싱 + empId/deptCd 추출
// 2) REQUIRED_MISSING / EMP_ID_FORAMT 검증
// 3) Connection 생성 + setAutoCommit(false)
// 4) inbox insert + SQLState 23000 중복(SKIP) 관리
// 5) account 생성 -> commit -> markSuccess()
// 6) SQLException 발생 시 rollback + 오류코드 분류
// 7) 별도 connection으로 inbox 실패 기록
// 8) markFail()
// 9) finally에서 autoCommit 복구 + close
private String processOne(Outbox o) {
    long startMs = System.currentTimeMillis();
    Connection conn = null;
    try {
        Map<String, Object> body = JsonUtil.parse(o.payload);
        String empId  = JsonUtil.getString(body, "empId",  "");
        String deptCd = JsonUtil.getString(body, "deptCd", "");

        if (empId.isEmpty() || deptCd.isEmpty()) throw new SQLException("REQUIRED_MISSING");
        if (!empId.matches("E\\d{8}\\d{3}"))     throw new SQLException("EMP_ID_FORMAT");

        conn = DBManager.getGroupwareConnection();
        conn.setAutoCommit(false);

        try {
            inboxDAO.insertInTx(conn, o.ifId, o.txNo, o.payload, "S", 0, null);
        } catch (SQLException ex) {
            if ("23000".equals(ex.getSQLState())) {
                conn.rollback();
                outboxDAO.markSuccess(o.outboxId);
                return "SKIP: 멱등 중복 (이미 발급된 계정)";
            }
            throw ex;
        }

        accountDAO.createAccountInTx(conn, empId, deptCd, o.txNo);

        conn.commit();
        outboxDAO.markSuccess(o.outboxId);
        return String.format("OK (account=%s, %dms)", empId, System.currentTimeMillis() - startMs);
```

1. Payload JSON을 파싱하여 empId, deptCd를 추출하고 필수값 및 사번 형식 검증

2. DB 트랜잭션을 시작한 뒤 Inbox 기록을 저장하고, 중복(SQLState 23000) 발생 시 SKIP 처리

3. 계정 생성 작업을 수행한 후 commit 및 markSuccess()를 호출하여 성공 상태로 완료

03 RECV - AccountBatchServlet-2(예외 처리)

```
        } catch (SQLException e) {
            String msg = e.getMessage();
            String errCode;
            if      (msg != null && msg.startsWith("DEPT_NOT_FOUND")) errCode = "E30";
            else if ("REQUIRED_MISSING".equals(msg))                  errCode = "E10";
            else if ("EMP_ID_FORMAT".equals(msg))                     errCode = "E32";
            else                                                      errCode = "E90";

            try { if (conn != null) conn.rollback(); } catch (SQLException ignored) {}

            try (Connection c2 = DBManager.getGroupwareConnection()) {
                c2.setAutoCommit(true);
                try {
                    inboxDAO.insertInTx(c2, o.ifId, o.txNo, o.payload, "F",
                            (int)(System.currentTimeMillis() - startMs),
                            errCode + ": " + msg);
                } catch (SQLException ignored) { }
            } catch (SQLException ignored) {}

            try { outboxDAO.markFail(o.outboxId, errCode + ": " + msg); }
            catch (SQLException ignored) {}
            return "FAIL " + errCode + ": " + msg;

        } catch (Exception e) {
            e.printStackTrace();
            try { if (conn != null) conn.rollback(); } catch (SQLException ignored) {}
            try { outboxDAO.markFail(o.outboxId, "E99: " + e.getMessage()); }
            catch (SQLException ignored) {}
            return "FAIL E99: " + e.getMessage();

        } finally {
            if (conn != null) {
                try { conn.setAutoCommit(true); } catch (SQLException ignored) {}
                DBManager.close(conn);
            }
        }
    }
}
```

1. SQLException 발생 시 오류 유형별 코드(E10, E30, E32, E90)를 분류하고 트랜잭션 rollback

2. 별도 Connection으로 Inbox 실패 로그를 기록하고 markFail()을 통해 Outbox 실패 상태 저장

3. finally에서 autoCommit 설정을 복구하고 DB Connection 종료

03 RECV - AccountDAO

```
    // TODO :
    // 1) SQL_CHECK_DEPT로 부서 존재 확인
    // 2) 초기 비밀번호 = empId + DBManager.getInitPasswordSuffix()
    // 3) SHA-256 해시 생성
    // 4) SQL_INSERT_ACCOUNT 실행 (empId, pwdHash, deptCd)
    // 5) SQL_INSERT_HIST 실행 (empId, txNo)

    // 부서 존재 확인
    try (PreparedStatement pstmt = conn.prepareStatement(SQL_CHECK_DEPT)) {
        pstmt.setString(1, deptCd);
        try (ResultSet rs = pstmt.executeQuery()) {
            if (!rs.next()) throw new SQLException("DEPT_NOT_FOUND:" + deptCd);
        }
    }

    // 비밀번호 해시 = SHA-256(empId + 초기비번 suffix)
    String pwdPlain = empId + DBManager.getInitPasswordSuffix();
    String pwdHash  = HashUtil.sha256(pwdPlain);

    try (PreparedStatement pstmt = conn.prepareStatement(SQL_INSERT_ACCOUNT)) {
        pstmt.setString(1, empId);
        pstmt.setString(2, pwdHash);
        pstmt.setString(3, deptCd);
        pstmt.executeUpdate();
    }

    try (PreparedStatement pstmt = conn.prepareStatement(SQL_INSERT_HIST)) {
        pstmt.setString(1, empId);
        pstmt.setString(2, txNo);
        pstmt.executeUpdate();
    }

}
```

1. SQL 조회를 통해 입력된 deptCd의 부서 존재 여부를 확인하고, 없으면 DEPT_NOT_FOUND 예외 발생

2. empId와 초기 비밀번호 suffix를 조합하여 기본 비밀번호를 생성한 뒤 SHA-256 해시로 암호화

3.계정 정보를 ACCOUNT 테이블에 저장하고, 생성 이력을 HIST 테이블에 기록

03 RECV - AccountDAO

```
/** 모니터링 — 계정 목록 (pwd_hash 노출 X) */
public List<Map<String, Object>> listAccounts() throws SQLException {

    // TODO :
    // SQL_LIST 실행
    // Connection / PrepareedStatement / ResultSet 생성
    // 조회 결과를 List<Map<String, Object>>로 변환
    // pwd_hash 제외(account_id, dept_cd, status, create_dt만 저장)
    // 결과 List 반환

    List<Map<String, Object>> out = new ArrayList<>();
    try (Connection conn = DBManager.getGroupwareConnection();
         PreparedStatement pstmt = conn.prepareStatement(SQL_LIST);
         ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            Map<String, Object> row = new LinkedHashMap<>();
            row.put("accountId", rs.getString("account_id"));
            row.put("deptCd",    rs.getString("dept_cd"));
            row.put("status",    rs.getString("status"));
            row.put("createDt",  rs.getTimestamp("create_dt"));
            out.add(row);
        }
    }
    return out;
  }
}
```

1. DB 연결 후 SQL_LIST를 실행하여 계정 목록 데이터 조회

2. 조회 결과(ResultSet) List<Map<String,Object>> 형태로 변환하여 필요한 컬럼(account_id, dept_cd, status, create_dt)만 저장

3. 보안상 pwd_hash는 제외하고, 최종 계정 목록 리스트 반환

03 RECV - InboxDAO

```java
/**
 * if_inbox DAO — tx_no UNIQUE 멱등 보장
 */
public class InboxDAO {

    private static final String SQL_INSERT =
            "INSERT INTO if_inbox (if_id, tx_no, payload, status, proc_ms, err_msg) " +
            "VALUES (?, ?, ?, ?, ?, ?)";

    private static final String SQL_LIST =
            "SELECT inbox_id, if_id, tx_no, status, proc_ms, recv_dt, err_msg " +
            "FROM if_inbox ORDER BY inbox_id DESC LIMIT 100";

    public void insertInTx(Connection conn, String ifId, String txNo, String payload,
                           String status, int procMs, String errMsg) throws SQLException {

        // TODO :
        // SQL_INSERT PreparedStatement 생성
        // ifId, txNo, payLoad, status, procMs, errMsg 값 바인딩
        // executeUpdate() 실행
        // 외부 트랜잭션 안에서 수행

        try (PreparedStatement pstmt = conn.prepareStatement(SQL_INSERT)) {
            pstmt.setString(1, ifId);
            pstmt.setString(2, txNo);
            pstmt.setString(3, payload);
            pstmt.setString(4, status);
            pstmt.setInt   (5, procMs);
            pstmt.setString(6, errMsg);
            pstmt.executeUpdate();
        }
    }
```

1. SQL_INSERT용 PreparedStatement를 생성하여 Inbox 저장 쿼리를 준비

2. ifId, txNo, payload, status, procMs, errMsg 값을 바인딩한 뒤 executeUpdate()로 INSERT를 수행

3. 해당 작업은 자체 commit 없이 외부 트랜잭션 (Connection) 내부에서 실행되어 멱등 처리에 사용

03 RECV - InboxDAO

```
    public List<Inbox> listAll() throws SQLException {

        //TODO :
        // SQL_LIST 실행
        // Connection / PreparedStatement / ResultSet 생성
        // 조회 결과를 Inbox 객체에 매핑
        // List<Inbox>에 저장 후 반환

        List<Inbox> list = new ArrayList<>();
        try (Connection conn = DBManager.getGroupwareConnection();
             PreparedStatement pstmt = conn.prepareStatement(SQL_LIST);
             ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                Inbox in = new Inbox();
                in.inboxId = rs.getLong("inbox_id");
                in.ifId    = rs.getString("if_id");
                in.txNo    = rs.getString("tx_no");
                in.status  = rs.getString("status");
                in.procMs  = rs.getInt("proc_ms");
                in.recvDt  = rs.getTimestamp("recv_dt");
                in.errMsg  = rs.getString("err_msg");
                list.add(in);
            }
        }
        return list;
    }

    public static class Inbox {
        public long      inboxId;
        public String    ifId;
        public String    txNo;
        public String    status;
        public int       procMs;
        public Timestamp recvDt;
        public String    errMsg;

        public long      getInboxId() { return inboxId; }
        public String    getIfId()    { return ifId; }
        public String    getTxNo()    { return txNo; }
        public String    getStatus()  { return status; }
        public int       getProcMs()  { return procMs; }
        public Timestamp getRecvDt()  { return recvDt; }
        public String    getErrMsg()  { return errMsg; }
    }
}
```

1. DB 연결 후 SQL_LIST를 실행하여 Inbox 테이블 데이터 조회

2. 조회된 ResultSet 데이터를 Inbox 객체(inboxId, ifId, txNo, status, procMs, recvDt, errMsg)에 매핑

3. 매핑된 객체를 List<Inbox>에 저장하여 전체 Inbox 목록으로 반환

04 SEND-Employee 테이블 구현

```java
public class Employee {

    // 6개 필드 선언
    private String empId;        // VARCHAR(12) E + YYYYMMDD + 3자리
    private String empName;      // VARCHAR(50) UTF-8
    private String deptCd;       // CHAR(4)
    private String position;     // VARCHAR(20)
    private String hireDt;       // CHAR(8) YYYYMMDD
    private String email;        // VARCHAR(100)

    // 기본 생성자 구현
    public Employee() {}

    // 전체 생성자 구현
    public Employee(String empId, String empName, String deptCd,
                    String position, String hireDt, String email) {
        this.empId    = empId;
        this.empName  = empName;
        this.deptCd   = deptCd;
        this.position = position;
        this.hireDt   = hireDt;
        this.email    = email;
    }

    // getter / setter 메서드 구현
    public String getEmpId()             { return empId; }
    public void   setEmpId(String v)     { this.empId = v; }
    public String getEmpName()           { return empName; }
    public void   setEmpName(String v)   { this.empName = v; }
    public String getDeptCd()            { return deptCd; }
    public void   setDeptCd(String v)    { this.deptCd = v; }
    public String getPosition()          { return position; }
    public void   setPosition(String v)  { this.position = v; }
    public String getHireDt()            { return hireDt; }
    public void   setHireDt(String v)    { this.hireDt = v; }
    public String getEmail()             { return email; }
    public void   setEmail(String v)     { this.email = v; }
}
```

1. Employee 클래스 내부에 6개의 필드를 private로 선언하였습니다.

2. 기본 생성자와 모든 인자를 받는 전체 생성자를 구현하였습니다.

3. 각 인자를 받기 위한 getter / setter 메서드를 구현하였습니다.

04 SEND-EmployeeDAO 테이블 구현

```java
public long createEmployeeWithOutbox(Employee emp) throws SQLException {
    // TODO: 위 7단계 구현
    Connection conn = null;
    try {
        // DB연결 후 수동으로 커밋 설정
        conn = DBManager.getHrmConnection();
        conn.setAutoCommit(false);

        // 사원 정보 데이터 삽입
        try (PreparedStatement pstmt = conn.prepareStatement(SQL_INSERT_EMP)) {
            pstmt.setString(1, emp.getEmpId());
            pstmt.setString(2, emp.getEmpName());
            pstmt.setString(3, emp.getDeptCd());
            pstmt.setString(4, emp.getPosition());
            pstmt.setString(5, emp.getHireDt());
            pstmt.setString(6, emp.getEmail());
            pstmt.executeUpdate();
        }

        // Json으로 변환
        String payload = toPayloadJson(emp);
        long outboxId;
```

1. DBManager를 통해 DB와의 연결을 가져옵니다. setAutoCommit(false)를 호출하여 자동 커밋을 끕니다.

2. 파라미터로 받은 사원(emp) 객체의 정보(ID, 이름, 부서, 직급, 고용일자, 이메일)를 사용하여 사원 테이블에 데이터를 삽입(INSERT)합니다.

3. 사원 정보를 JSON 형태의 문자열로 변환합니다.

04 SEND-EmployeeDAO 테이블 구현

```
        // Outbox 테이블(SQL_INSERT_OUTBOX)에 데이터를 삽입
        try (
                PreparedStatement pstmt = conn.prepareStatement(
                SQL_INSERT_OUTBOX, Statement.RETURN_GENERATED_KEYS)
            )
        {
            pstmt.setString(1, IF_ID);
            pstmt.setString(2, emp.getEmpId());    // 멱등키
            pstmt.setString(3, payload);
            pstmt.executeUpdate();
            // ID 추출
            try (ResultSet keys = pstmt.getGeneratedKeys()) {
                outboxId = keys.next() ? keys.getLong(1) : -1L;
            }
        }

        // 최종 커밋
        conn.commit();
        return outboxId;

    }
    // 예외 발생시 원상복구
    catch (SQLException e) {
        if (conn != null) try { conn.rollback(); } catch (SQLException ignored) {}
        throw e;
    }
    // 커밋 설정 초기화 및 DB 연결 반납
    finally {
        if (conn != null) {
            try { conn.setAutoCommit(true); } catch (SQLException ignored) {}
            DBManager.close(conn);
        }
    }
```

1. 이벤트 발행을 위해 Outbox 테이블에 (SQL_INSERT_OUTBOX)데이터를 삽입합니다.

2. 방금 Outbox 테이블에 데이터가 들어가면서 자동으로 생성된 시퀀스 번호(또는 Auto Increment ID)를 ResultSet을 통해 조회하여 outboxId 변수에 담습니다. 만약 가져오지 못했다면 -1L을 할당합니다.

3. 최종적으로 수동으로 커밋해주고 데이터를 반환합니다.

4.try 블록 안에서 어떠한 예외라도 발생하면 conn.rollback()을 호출하여 사원 추가와 Outbox 추가 작업을 모두 취소하고 finally는 무조건 실행되기에 커밋 설정을 다시 원래대로 돌려놓고, 데이터베이스 연결(conn)을 닫아 반납합니다.

04 SEND-EmpRegistServlet 테이블 구현

```
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
    // TODO: 위 8단계 구현
    // HTTP 요청 확인
    System.out.println("POST /send/emp");

    // 파라미터 받기 후 공백 제거
    String empName  = trim(req.getParameter("empName"));
    String deptCd   = trim(req.getParameter("deptCd"));
    String position = trim(req.getParameter("position"));
    String email    = trim(req.getParameter("email"));

    // 유효성 검증
    if (empName.isEmpty() || deptCd.isEmpty() || position.isEmpty()) {
        forward(req, resp, "FAIL", "E10: 필수값(empName/deptCd/position) 누락", null);
        return;
    }
    if (empName.length() > 50) {
        forward(req, resp, "FAIL", "E10: empName 길이 초과 (50)", null);
        return;
    }
    if (!deptCd.matches("D\\d{3}")) {
        forward(req, resp, "FAIL", "E10: deptCd 형식 오류 (Dnnn)", null);
        return;
    }
    if (!ALLOWED_POSITIONS.contains(position)) {
        forward(req, resp, "FAIL", "E31: 미지원 직급 (" + position + ")", null);
        return;
    }
```

1. 서버 콘솔에 로그를 남겨 요청이 잘 들어왔는지 확인합니다.

2. 화면에서 사용자가 입력한 값(이름, 부서코드, 직급, 이메일)을 꺼내옵니다. trim() 메서드를 이용해 앞뒤 불필요한 공백을 제거하합니다.

3. 필수 값이 비어있지 않은지, 이름이 너무 길지 않은지 (50자 제한),부서 코드가 형식(D로 시작하고 뒤에 숫자 3자리, 예: D001)에 맞는지, 허용된 직급 리스트(ALLOWED_POSITIONS)에 포함되는지에 대한 유효성 검증을 진행합니다.

04 SEND-EmpRegistServlet 테이블 구현

```
    // 비즈니스 데이터(사원 정보) 생성
    String hireDt = new SimpleDateFormat("yyyyMMdd").format(new Date());
    String empId  = "E" + hireDt + String.format("%03d", new Random().nextInt(1000));
    // 입력받은 데이터와 생성된 데이터를 묶어 하나의 객체 생성
    Employee emp = new Employee(empId, empName, deptCd, position, hireDt, email);

    try {
        // empDAO의 메서드 호출
        long outboxId = empDAO.createEmployeeWithOutbox(emp);
        // 성공 메시지 구현
        String msg = String.format(
            "입사 등록 완료. empId=%s, outboxId=%d (계정은 배치 처리 후 발급됩니다)",
            empId, outboxId);
        forward(req, resp, "OK", msg, empId);
    }
    // 에러 제어용 예외 처리
    catch (SQLException e) {
        e.printStackTrace();
        forward(req, resp, "FAIL", "E90 DB 오류: " + e.getMessage(), null);
    }
}

private static String trim(String s) { return s == null ? "" : s.trim(); }

// 뷰(JSP) 화면으로 데이터 전달 및 이동
private void forward(HttpServletRequest req, HttpServletResponse resp,
                     String status, String message, String empId)
        throws ServletException, IOException {
    req.setAttribute("status",  status);
    req.setAttribute("message", message);
    req.setAttribute("empId",   empId);
    req.getRequestDispatcher("/send/result.jsp").forward(req, resp);
}
```

1. 오늘 날짜를 기준으로 입사일자(hireDt)를 구하고, 이를 조합하여 고유한 사원 번호(empId, 예: E20260526789)를 생성합니다.
입력받은 데이터와 생성된 데이터를 묶어 하나의 Employee 객체로 조립합니다.

2. 앞서 구현했던 DAO의 메서드를 호출합니다.사원 정보와 트랜잭션 아웃박스(Outbox) 데이터를 DB에 한 번에 안전하게 저장하고, 생성된 outboxId를 받아옵니다. DB 저장이 성공하면 화면에 띄워줄 성공 메시지를 만들고, 상태 값을 "OK"로 지정하여 화면 이동 함수(forward)를 호출합니다.

3. DB 저장) 도중 중복 키 에러나 연결 끊김 등 An SQLException이 발생하면 에러 내용을 서버 로그에 출력(printStackTrace)하고, 사용자에게는 "FAIL" 상태와 에러 메시지를 전달하도록 구성합니다.

4. 결과 상태(status), 메시지(message), 사원번호(empId)를 request 객체에 보관(저장)합니다. 이렇게 저장해야 이동한 화면에서 이 값들을 꺼내 쓸 수 있습니다. 최종적으로 /send/result.jsp 화면으로 요청을 넘겨서(Forward), 사용자가 브라우저에서 깔끔하게 정돈된 결과 페이지를 볼 수 있도록 처리합니다.

04 SEND-OutboxDAO 테이블 구현

```java
public List<Outbox> fetchPending(int maxRetry, int limit) throws SQLException {
    /* TODO */
    // 저장 공간 준비
    List<Outbox> out = new ArrayList<>();
    // DB 연결 및 SQL 실행 준비
    try (Connection conn = DBManager.getHrmConnection();
         PreparedStatement pstmt = conn.prepareStatement(SQL_FETCH_PENDING)) {
        // 쿼리 파라미터 세팅
        pstmt.setInt(1, maxRetry);
        pstmt.setInt(2, limit);
        // SQL 쿼리 실행 및 결과셋(ResultSet) 획득
        try (ResultSet rs = pstmt.executeQuery()) {
            // 객체 변환 후 데이터 추출
            while (rs.next()) {
                Outbox o = new Outbox();
                o.outboxId = rs.getLong("outbox_id");
                o.ifId     = rs.getString("if_id");
                o.txNo     = rs.getString("tx_no");
                o.payload  = rs.getString("payload");
                o.retryCnt = rs.getInt("retry_cnt");
                out.add(o);
            }
        }
    }
    // 결과 반환하고 자원 해제
    return out;
}
```

1. DB에서 조회한 아직 처리되지 않은(Pending 상태인) Outbox 레코드들을 담아서 반환할 빈 리스트(out)를 생성합니다.

2. Connection과 PreparedStatement를 생성하여 자원 할당 및 자동 반납을 설정합니다.

3. 쿼리 파라미터를 세팅하고 데이터베이스에 쿼리를 던져 조건에 맞는 데이터를 조회하고, 그 결과 데이터가 담긴 ResultSet을 받아옵니다.

4. rs.next()를 통해 조회된 테이블의 행(Row)을 한 줄씩 차례대로 읽어 내려갑니다. DB의 컬럼값들(outbox_id, if_id, tx_no, payload, retry_cnt)을 자바 객체인 Outbox 내부 필드에 하나씩 매칭하여 저장합니다. 데이터가 채워진 Outbox 객체를 1단계에서 만든 out 리스트에 차곡차곡 추가합니다.

5. DB에서 조회하여 객체로 변환 완료된 Outbox 데이터 리스트(out)를 호출한 곳으로 반환합니다.

04 SEND-OutboxDAO 테이블 구현

```java
public int markSuccess(long outboxId) throws SQLException {
    /* TODO */
    // DB 연결 및 SQL 실행 준비
    try (Connection conn = DBManager.getHrmConnection();
         PreparedStatement pstmt = conn.prepareStatement(SQL_MARK_SUCCESS)) {
        // 쿼리 파라미터 세팅
        pstmt.setLong(1, outboxId);
        // 업데이트 실행 및 결과 반환
        return pstmt.executeUpdate();
    }
}
```

1. Connection과 PreparedStatement 객체를 생성합니다. 성공 상태로 업데이트하는 쿼리문인 SQL_MARK_SUCCESS를 실행할 준비를 합니다.

2. 쿼리문 내부의 첫 번째 자리에 매개변수로 받은 outboxId를 매핑합니다.

3. executeUpdate()를 호출하여 데이터베이스의 데이터를 수정합니다.

04 SEND-OutboxDAO 테이블 구현

```java
public int markFail(long outboxId, String errMsg) throws SQLException {
    /* TODO */
    // DB 연결 및 SQL 실행 준비
    try (Connection conn = DBManager.getHrmConnection();
         PreparedStatement pstmt = conn.prepareStatement(SQL_MARK_FAIL)) {
        // 쿼리 파라미터 세팅
        pstmt.setString(1, errMsg);
        pstmt.setLong  (2, outboxId);
        // 업데이트 실행 및 결과 반환
        return pstmt.executeUpdate();
    }
}
```

1. Connection과 PreparedStatement 객체를 생성합니다. 실패 상태와 에러 메시지를 업데이트하는 쿼리문인 SQL_MARK_FAIL을 실행할 준비를 합니다.

2. 쿼리문 내부의 첫 번째 자리에 실패 원인이 담긴 예외 메시지(errMsg)를 넣습니다. 두 번째 자리에는 대상이 되는 특정 메시지의 고유 번호(outboxId)를 지정합니다.

3. executeUpdate()를 호출하여 데이터베이스의 데이터를 수정합니다.

04 SEND-OutboxDAO 테이블 구현

```java
public int reopen(long outboxId) throws SQLException {
    /* TODO */
    // DB 연결 및 SQL 실행 준비
    try (Connection conn = DBManager.getHrmConnection();
         PreparedStatement pstmt = conn.prepareStatement(SQL_REOPEN)) {
        // 쿼리 파라미터 세팅
        pstmt.setLong(1, outboxId);
        // 업데이트 실행 및 결과 반환
        return pstmt.executeUpdate();
    }
}
```

1. 데이터베이스 커넥션을 가져오고 재오픈 쿼리 (SQL_REOPEN)를 수행할 준비를 합니다.

2. 쿼리문 내부의 첫 번째 자리에 아웃박스 ID(outboxId)를 매핑합니다.

3. executeUpdate()를 호출하여 데이터베이스의 데이터를 수정합니다.

04 SEND-OutboxDAO 테이블 구현

```java
public List<Outbox> listAll() throws SQLException {
    /* TODO */
    // 저장 공간 준비
    List<Outbox> out = new ArrayList<>();
    // DB 연결, SQL 실행 및 결과셋(ResultSet) 동시 준비
    try (Connection conn = DBManager.getHrmConnection();
         PreparedStatement pstmt = conn.prepareStatement(SQL_LIST_ALL);
         ResultSet rs = pstmt.executeQuery()) {
        // 전체 데이터 추출
        while (rs.next()) {
            Outbox o = new Outbox();
            o.outboxId = rs.getLong("outbox_id");
            o.ifId     = rs.getString("if_id");
            o.txNo     = rs.getString("tx_no");
            o.status   = rs.getString("status");
            o.retryCnt = rs.getInt("retry_cnt");
            o.regDt    = rs.getTimestamp("reg_dt");
            o.procDt   = rs.getTimestamp("proc_dt");
            o.errMsg   = rs.getString("err_msg");
            out.add(o);
        }
    }
    // 결과 반환
    return out;
}
```

1. DB에서 조회할 모든 Outbox 데이터를 담아서 반환할 빈 리스트(out)를 생성합니다.

2. Connection과 PreparedStatement 객체를 생성합니다. 또한 rs객체를 선언하여 파라미터가 없는 전체 조회용 SQL인 SQL_LIST_ALL(예: SELECT * FROM outbox)을 실행하여 모든 데이터를 rs 객체에 담습니다.

3. 데이터베이스에 적재된 모든 줄(Row)을 맨 위부터 마지막 줄까지 while 루프를 돌며 한 줄씩 읽습니다. 변환된 객체들을 out 리스트에 추가합니다.

4. 모든 데이터를 담은 리스트를 반환합니다.